DIGITALHOUSE CLASE EN VIVO 3

ARCA | AGENCIA DE RECAUDACIÓN Y CONTROL ADUANERO

Pipelines con Apache NiFi

Clase en vivo 3 · 120 minutos

Programa: Arquitectura de Datos, Gobierno y Orquestación — ARCA

Escaneá el QR para registrar tu asistencia

Me presento...

1. Directora General en The eLabs: transformación digital e integración de IA.
2. Host del podcast Hack The Future: estrategia y liderazgo, no solo algoritmos.
3. Docente en la Maestría en Ciencia de Datos (Univ. Austral), ITBA y Di Tella.



Apasionada por aplicar IA para mejorar la productividad y las decisiones.

Lo que dejó la clase 2

1 Un contrato

Parámetros, resultado y errores, escritos antes del cuerpo.

2 Un solo COMMIT

Confirmar en el nivel que ve toda la operación.

3 Una bitácora

Qué pasó en cada corrida, aunque se revierta.

4 Idempotencia

Cargar dos veces deja el mismo estado.

Hoy miramos el tramo anterior: cómo llega el dato hasta el área de paso.

Qué vamos a lograr hoy

PROPÓSITO

Diseñar un flujo de ingestión que no pierda datos y deje a la vista lo que falla.

RESULTADO OBSERVABLE

Un pipeline dibujado con cuatro rutas (éxito, rechazo, reintento y cuarentena) y la decisión de qué herramienta usar.

A lo largo de la clase hay un ejercicio en cuatro etapas: diseñás el pipeline de ingestión de NexoServicios, paso a paso.

Un script suelto falla siempre igual

1 No se ve qué pasó

Si falla de madrugada, nadie se entera hasta que alguien lo nota.

2 No sabe reintentar

Una caída de cinco minutos del destino alcanza para perder el archivo del día.

3 No sabe frenar

Si el destino va lento, sigue empujando datos hasta desbordarlo.

4 Duplica o pierde datos

Tras un reintento manual, nadie sabe qué se cargó y qué no.

Son cuatro problemas de operación, no de programación.

QUÉ ES

Apache NiFi, en una frase

Una herramienta para mover datos entre sistemas de forma confiable y visible: se arma dibujando un flujo, y por él viajan los datos.

Flujo

El camino que recorre el dato.

Cola

Cada paso tiene su espera, con límite.

Trazabilidad

Qué le pasó a cada dato.

No analiza los datos: los lleva, los vigila y deja constancia.

De la NSA a la comunidad Apache

Antes de 2014

Nace en la NSA como «Niagara Files», para mover datos entre sistemas muy distintos.

2014

Se libera a la Apache Software Foundation.

2015

Pasa a ser un proyecto de primer nivel de Apache.

Hoy

Software libre, mantenido por la comunidad, con versiones comerciales con soporte.

Referencia: nifi.apache.org, la documentación oficial del proyecto.

Cinco piezas y ya se lee un flujo

1

FlowFile

El dato que viaja: contenido más etiquetas.

2

Procesador

Una caja que hace una sola cosa.

3

Conexión

La flecha, con una cola y un límite.

4

Relación

Cada salida con nombre: éxito, falla…

5

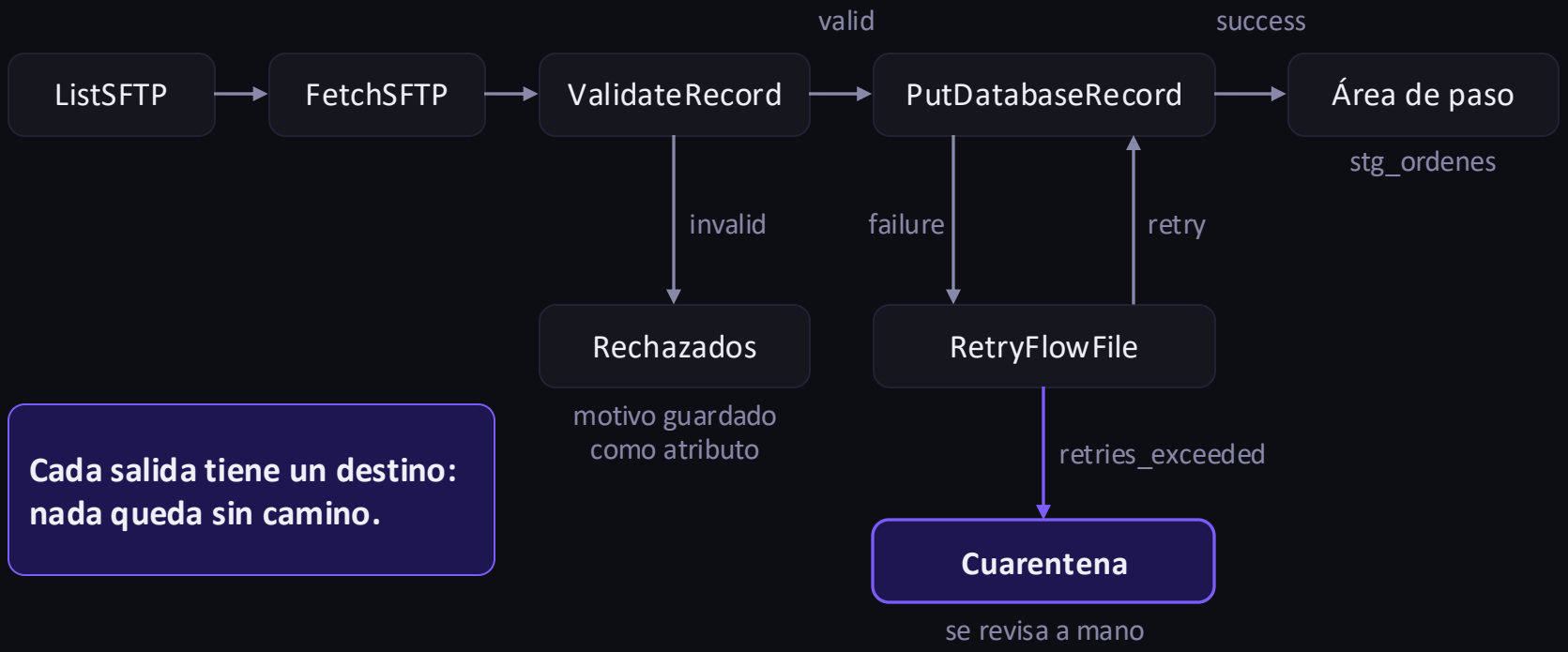
Grupo de procesos

Una carpeta que ordena un sub-flujo.

Si una salida no tiene destino, el procesador no arranca: NiFi te obliga a decidir qué hacer con cada falla.

CÓMO SE VE

Un flujo, de punta a punta



Cada salida tiene un destino: nada queda sin camino.

Cinco gestos para usar NiFi

1

Arrastrar

Llevás un procesador de la barra al lienzo.

2

Configurar

Doble clic: elegís sus propiedades, como la carpeta.

3

Conectar

Unís una salida con el paso siguiente: NiFi crea la cola.

4

Arrancar

Play: los datos empiezan a moverse.

5

Mirar

Contadores e historial: qué pasó con cada dato.

Casi todo se hace en pantalla: el código solo aparece para reglas puntuales, como una consulta SQL.

EN VIVO

El lienzo y el camino feliz

QUÉ VAMOS A VER

- El lienzo y los siete procesadores del flujo.
- Las propiedades de un procesador: de qué carpeta lee.
- Una consulta SQL que separa lo válido de lo rechazado.
- Un archivo sano que llega hasta el destino.

QUÉ MIRAR

- Los contadores de cada conexión, que se mueven.
- Cuánto tarda el dato en llegar.
- Qué reglas son de negocio, y de quién son.

Para pensar: **¿Quién decide que el monto máximo es 50.000?**

Éxito, rechazo, reintento y cuarentena

Éxito

CUÁNDO

El dato es válido y sigue hasta el destino.

QUIÉN ACTÚA

Nadie: es automático.

Rechazo

CUÁNDO

El dato está mal y reintentar no lo arregla.

QUIÉN ACTÚA

El dueño del dato, con el motivo guardado.

Reintento

CUÁNDO

Falló algo pasajero, como el destino caído.

QUIÉN ACTÚA

El sistema, con pausa y tope de intentos.

Cuarentena

CUÁNDO

Se agotaron los intentos o no se sabe qué pasó.

QUIÉN ACTÚA

Una persona, con una alerta.

Monitorear no es recuperar: la alerta avisa, la ruta resuelve.

EN VIVO

Las rutas, en vivo

QUÉ VAMOS A VER

- Un archivo con 6 filas malas de 20: rechazo, con su motivo.
- Un destino caído: el dato se reintenta solo, y carga cuando se arregla.
- Un destino que nadie arregla: el dato va a la cuarentena.

QUÉ MIRAR

- A qué carpeta va cada dato.
- Cuántas veces reintenta, y cada cuánto.
- Quién actuó en cada caso: el sistema o una persona.

Para pensar: **¿Reintentar esas seis filas las arreglaría?**

Dónde se rompen los diseños

Error	Qué provoca	Cómo evitarlo
Reintentar un dato inválido	Bloquea la cola para siempre	Mandarlo a rechazo, con el motivo
Mandar todo a cuarentena	La revisión se vuelve imposible	Separar falla de dato, técnica y desconocida
Cola sin límite	Se desborda un destino lento	Definir la contrapresión
Confiar solo en las alertas	Se sabe que falló, no cómo se recupera	Diseñar la ruta de recuperación
Destino que no tolera duplicados	Un reintento carga dos veces	Destino idempotente, como en la clase 2
Reglas de negocio escondidas	Nadie sabe quién las decide	El dueño del dato define el esquema

Cada error se evita decidiendo, antes de construir, qué pasa con cada falla.

Diseñar bien y reconocer lo que falla

Buenas prácticas de diseño

- Decidir, antes de construir, qué pasa con cada falla.
- Un límite en cada cola, para proteger al destino.
- Reintentos con pausa y con tope; después, cuarentena.
- Un destino idempotente: cargar dos veces no duplica.
- Dejar evidencia: motivo del rechazo y trazabilidad.
- Flujos ordenados en grupos, con nombres claros y los secretos en parámetros.

Errores que vas a encontrar

El procesador no arranca
→ Hay una salida sin destino.

La cola se llena y el flujo se frena
→ El destino va lento o está caído.

Un dato se queda esperando
→ Falló y está en pausa antes de reintentarse.

Aparecen duplicados
→ El archivo llegó dos veces y el destino no es idempotente.

Un archivo no se vuelve a traer
→ La lectura recuerda que ya lo vio.

Corre, pero nadie sabe qué pasó
→ No se guardó el motivo de los rechazos.

EN VIVO

Trazabilidad y cola llena

QUÉ VAMOS A VER

- El historial de un dato, de punta a punta.
- Cómo reprocesar un dato desde el historial.
- Una cola llena que frena al paso anterior.

QUÉ MIRAR

- Los eventos de cada dato, con su hora.
- El color de la conexión cuando se llena.
- Qué se frena y qué sigue andando.

Para pensar: **En un script de Python, ¿qué tendrías que escribir para tener esto?**

Qué tendrías que escribir y mantener vos

Necesidad	En un script de Python	En NiFi
Reintentos con pausa	Un bucle con una espera y un tope	Una propiedad del procesador
Cola con límite	Una estructura propia y su lógica de frenado	Contrapresión en cada conexión
Qué archivo ya procesé	Un archivo o tabla de estado propia	El estado del procesador de lectura
Qué le pasó a este dato	Logs que armás vos	La trazabilidad de cada dato
Qué hacer con cada falla	Un bloque de excepción que se puede olvidar	Cada salida necesita un destino
Que otra persona lo mantenga	Leer el código	Leer el dibujo

Todo se puede hacer en Python: cambia quién lo escribe, lo prueba y lo mantiene.

Qué hay más allá de NiFi

Herramienta	Qué resuelve	Cuándo la elegiría
Airflow, Dagster, Prefect	Orquestan tareas por lotes, en código	Importa el cuándo y el orden
Airbyte, Fivetran	Conectores administrados hacia un almacén	Todo es SaaS y querés cero mantenimiento
Kafka y Kafka Connect	Transmiten eventos y conectan sistemas	Kafka es la columna vertebral
StreamSets, Talend, Informatica	Suites de integración comerciales	Hace falta soporte y gobierno corporativo
Azure Data Factory, AWS Glue	Lo mismo, administrado en la nube	Todo vive en una sola nube
Spark, dbt, SQL	Transforman datos que ya están adentro	El trabajo es transformar, no mover

Se combinan: no compiten en el mismo nivel.

Cada familia resuelve otra parte

Mover y enrutar

Que el dato llegue y se sepa qué le pasó.

NiFi, StreamSets, Airbyte

Orquestar

Que cada tarea corra a su hora y en su orden.

Airflow, Dagster, Prefect

Transmitir eventos

Que muchos sistemas lean los mismos eventos.

Kafka

Transformar a escala

Que el dato quede listo para analizar.

Spark, dbt, SQL

NiFi ingiere; un orquestador coordina; Kafka transmite; Spark y dbt transforman.

Elegir NiFi, o no

Elegir NiFi cuando…

- Hay muchas fuentes distintas: archivos, SFTP, APIs, bases.
- Importa saber qué le pasó a cada dato.
- El entorno es propio o regulado.
- Alguien que no programa tiene que entender el flujo.

No elegirlo cuando…

- La transformación es compleja y pesada.
- Es una tarea única y simple.
- Hace falta orquestar con calendario y dependencias.
- Nadie puede operar un servicio propio.

Un script alcanza para una tarea simple; NiFi se justifica cuando hay que operar.

Cuatro preguntas para decidir

1 ¿Cuántas fuentes y de qué tipo?

Mucha variedad inclina a NiFi.

2 ¿Cuánto importa la trazabilidad?

Si hay que saber qué le pasó a cada dato, NiFi.

3 ¿Quién lo va a operar?

Sin nadie que opere un servicio, mirar opciones administradas.

4 ¿Mover o transformar?

Si el trabajo es transformar, otra herramienta.

Las cuatro respuestas, juntas, justifican la herramienta.

CÓMO LO VAMOS A VER

Sin instalar nada

1 En pantalla

Quien dicta la clase muestra un flujo funcionando.

DEMO EN VIVO!

2 En el lienzo

Cada equipo diseña su pipeline con tarjetas, sin instalar nada.

3 Contra Python

Cada decisión se compara con lo que costaría escribirla.

Lo que se evalúa es el razonamiento, no la ejecución de una herramienta.

EJERCICIO A LO LARGO DE LA CLASE

Diseñá el pipeline de NexoServicios

1

El mapa

Después de «Cómo se piensa»

Fuentes, destino y qué tiene que llegar.

2

Las rutas

Después de «Las cuatro rutas»

Para cada falla: ruta y quién actúa.

3

La herramienta

Después de «Elegir NiFi, o no»

NiFi, Python u otra: elegí y justificá.

4

La defensa

Al final

Un equipo audita a otro.

Entregable: el pipeline dibujado, la tabla de fallas y la decisión de herramienta.

Siete casos, siete complicaciones

1

El archivo que no llega a horario

¿Todavía no llegó, o hoy no va a llegar?

2

El proveedor que corrige y reenvía

Ignorarlo pierde la corrección; cargarlo duplica.

3

La API con límite y caídas

Reintentar rápido bloquea; lento, no llega.

4

La base que no se puede molestar

Una consulta pesada la enlentece.

5

Datos sensibles en el camino

También aparecen en los errores y los rechazos.

6

El formato cambió y no avisaron

Parece válido, pero significa otra cosa.

7

Fin de mes: el volumen se multiplica

Empujar todo desborda; esperar, no llega.

La tabla de fallas

¿Qué falla?	¿A qué ruta va?	¿Quién actúa?	¿Qué evidencia deja?
Falla 1 de tu caso			
Falla 2 de tu caso			
Falla 3 de tu caso			
Falla 4 de tu caso			
Falla 5 de tu caso			

Cada caso trae sus cinco fallas: se completa durante la clase y es la base del pipeline que dibujás.

Qué me llevo y qué dejo escrito

CUATRO IDEAS PARA LLEVARSE

1. **Cada falla, un destino**
   Éxito, rechazo, reintento o cuarentena.
2. **La ruta depende del tipo de falla**
   Dato, técnica pasajera o desconocida.
3. **Elegir con criterio**
   Fuentes, trazabilidad, quién opera, mover o transformar.
4. **Monitorear no es recuperar**
   La alerta avisa; la ruta resuelve.

TICKET DE SALIDA · UNA FRASE POR CADA UNO

Decisión más sólida
La que mejor puede defender tu equipo.

Riesgo abierto
El que todavía no está controlado.

Próximo experimento
Qué probarías primero.

LO QUE VIENE

Hacia la clase 4

Con datos que llegan y fallas con destino, falta coordinar cuándo corre cada cosa y quién responde por ella.

Orquestación

Gobierno

Eso introduce la clase 4: orquestación y gobierno.

DIGITALHOUSE

ARCA | AGENCIA DE RECAUDACIÓN Y CONTROL ADUANERO

PREGUNTAS Y RESPUESTAS

Q&A

¿Qué dudas, casos o ideas quedaron?

¡Muchas gracias!

Laura Sol Franco · Digital House

Programa: Arquitectura de Datos, Gobierno y Orquestación — ARCA